

P1518R0

Stop overconstraining allocators in container deduction guides

Published Proposal, 2019-03-10

Authors:

[Arthur O'Dwyer](#)

[Mike Spertus](#)

Audience:

LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Current Source:

github.com/Quuxplusone/draft/blob/gh-pages/stack-ctad.bs

Current:

rawgit.com/Quuxplusone/draft/gh-pages/stack-ctad.html

Abstract

Discussion of flatmap's deduction guides revealed that the deduction guides for sequence containers and container adaptors are needlessly overconstrained, making use cases such as pmr containers unnecessarily difficult. We fix this.

Table of Contents

- 1 Stop overconstraining allocators that do not participate in deduction**
 - 1.1 The problem
 - 1.2 The explanation
 - 1.3 The solution

2 Stop deducing from allocators that should not participate in deduction

2.1 The problem

2.2 The explanation

2.3 The solution

2.3.1 Wording option 1

2.3.2 Wording option 2

2.4 Implementation note

§ 1. Stop overconstraining allocators that do not participate in deduction

§ 1.1. The problem

Consider this code:

```
std::pmr::monotonic_buffer_resource mr;
std::pmr::polymorphic_allocator<int> a = &mr;
std::pmr::vector<int> pv(a);

auto s1 = std::stack<int, std::pmr::vector<int>>(pv);
auto s2 = std::stack<int, std::pmr::vector<int>>(pv, a);
auto s3 = std::stack<int, std::pmr::vector<int>>(pv, &mr);

auto ds1 = std::stack(pv);
auto ds2 = std::stack(pv, a);
auto ds3 = std::stack(pv, &mr);
```

The initializers of `s1`, `s2`, and `s3` (which do not use CTAD) are all well-formed, as are the initializers of `ds1` and `ds2` (which do).

However, the natural and useful `ds3` is ill-formed, even though the `&mr` argument is irrelevant to determining the template arguments for `stack`! It seems clearly wrong that we give all the right information to unambiguously deduce the desired specialization of `stack` and then reject what would be a perfectly valid constructor invocation for that class. The allocator parameter's type does not *contribute* to class template argument deduction in these cases; it brings no new information, and therefore should be treated no differently than it is in the non-CTAD case. Indeed, we believe this is an oversight and should simply be fixed.

§ 1.2. The explanation

`stack` has one relevant deduction guide.

```
template<class Container, class Allocator>
stack(Container, Allocator) -> stack<typename Container::value_type, Co
```

This deduction guide satisfies the following constraints.

[[container.adaptors.general](#)] says:

A deduction guide for a container adaptor shall not participate in overload resolution if any of the following are true: ...

- It has an `Allocator` template parameter and a type that does not qualify as an allocator is deduced for that parameter.
- It has both `Container` and `Allocator` template parameters, and `uses_allocator_v<Container, Allocator>` is `false`.

To qualify as an allocator, a type `A` must have a nested typedef `A::value_type` ([[container.requirements.general](#)]/17). Since `std::pmr::monotonic_buffer_resource*` has no such nested typedef, `std::pmr::monotonic_buffer_resource*` doesn't qualify as an allocator. However, `uses_allocator_v<std::pmr::vector<int>, std::pmr::monotonic_buffer_resource*>` is `true`.

Because the type of `&mr` doesn't qualify as an allocator, the deduction guide drops out of overload resolution, failing deduction even though it has all the information needed to safely deduce the correct template arguments for `stack`.

§ 1.3. The solution

To fix all three container adaptors, modify [[container.adaptors.general](#)] as follows:

A deduction guide for a container adaptor shall not participate in overload resolution if any of the following are true:

- It has an `InputIterator` template parameter and a type that does not qualify as an input iterator is deduced for that parameter.
- It has a `Compare` template parameter and a type that qualifies as an allocator is deduced for that parameter.
- It has a `Container` template parameter and a type that qualifies as an allocator is deduced for that parameter.
- ~~It has an `Allocator` template parameter and a type that does not qualify as an allocator is deduced for that parameter.~~
- It has both `Container` and `Allocator` template parameters, and `uses_allocator_v<Container, Allocator>` is false.

Note: The rationale is that if the `Container` advertises that it can "use" the supplied allocator, we shouldn't interrogate it any more closely. Trust the `Container`'s specialization of `uses_allocator`.

Note: No container adaptor deduction guide has an `Allocator` template parameter without also having a `Container` template parameter.

§ 2. Stop deducing from allocators that should not participate in deduction

§ 2.1. The problem

Consider the following code:

```
std::pmr::monotonic_buffer_resource mr;
std::pmr::polymorphic_allocator<int> a = &mr;
std::pmr::vector<int> pv(a);

auto v1 = std::vector<int, std::pmr::polymorphic_allocator<int>>(pv);
auto v2 = std::vector<int, std::pmr::polymorphic_allocator<int>>(pv, a);
auto v3 = std::vector<int, std::pmr::polymorphic_allocator<int>>(pv, &mr);

auto dv1 = std::vector(pv);
auto dv2 = std::vector(pv, a);
auto dv3 = std::vector(pv, &mr);
```

The initializers of `v1`, `v2`, and `v3` (which do not use CTAD) are all well-formed, as are the initializers of `dv1` and `dv2` (which do).

But the initializer of `dv3` is ill-formed!

Again, we know from the `pv` argument that the correct type for the vector is `decltype(pv)`, i.e. `std::pmr::vector<int>`. Therefore we know what is the possible range of types for the allocator parameter. The allocator parameter's type does not *contribute* to class template argument deduction in these cases; it brings no new information, and therefore should be treated no differently than it is in the non-CTAD case.

The problem we see with `vector` also occurs for all other sequence containers, associative containers, and unordered associative containers.

§ 2.2. The explanation

`vector` has only one deduction guide, and it's not relevant to what happens here. Here, we end up looking at the implicit guide generated from the constructor

```
template<class T, class Allocator>
class vector {
    vector(const vector<T, Allocator>&, const Allocator&);
};
```

From the first parameter, we deduce `T=int` and `Allocator=std::pmr::polymorphic_allocator<int>`. From the second parameter, we deduce `Allocator=std::monotonic_buffer_resource*`. We've deduced conflicting types for `Allocator`, so deduction fails.

In this case, the second argument unnecessarily participates in deduction, and again unexpectedly prevents natural and useful code from working as desired.

§ 2.3. The solution

We present two approaches for changing the wording that are equivalent in effect.

§ 2.3.1. Wording option 1

If we were introducing the containers from scratch today, the natural approach would be for the constructors to indicate which arguments should be considered deducible, as is usual for function templates. For example, modify [\[deque.overview\]](#) as follows:

```
deque(deque&&);  
deque(const deque&, const Allocator&);  
deque(const deque&, const type_identity_t<Allocator>&);  
deque(deque&&, const Allocator&);  
deque(deque&&, const type_identity_t<Allocator>&);  
deque(initializer_list<T>, const Allocator& = Allocator());
```

(Notice that the constructor `deque(initializer_list<T>, const Allocator&)` is not changed, because its `Allocator` parameter actually does contribute to deduction: without that parameter's type, we would not know what the `deque`'s allocator type should be.)

If this is not deemed to cause compatibility issues, we like this approach.

§ 2.3.2. Wording option 2

First modify [\[sequence.reqmts\]/13](#) as follows:

A deduction guide for a sequence container shall not participate in overload resolution if it has an `InputIterator` template parameter and a type that does not qualify as an input iterator is deduced for that parameter, or if it has an `Allocator` template parameter and a type that does not qualify as an allocator is deduced for that parameter, or if it has an `Alloc` template parameter and `uses_allocator_v<X, Alloc>` is false where `X` is the deduced type of the sequence container.

(We think that change is not strictly necessary, but it clarifies our intent.)

Add one new deduction guide for each sequence container. For example, modify [\[deque.overview\]](#) as follows:

```

template<class InputIterator, class Allocator = allocator<iter-value-type<InputIterator>>
    deque(InputIterator, InputIterator, Allocator = Allocator())
    -> deque<iter-value-type<InputIterator>, Allocator>;

template<class T, class Allocator, class Alloc>
    deque(deque<T, Allocator>, Alloc)
    -> deque<T, Allocator>;

```

Then, modify [\[associative.reqmts\]](#)/15 as follows:

A deduction guide for an associative container shall not participate in overload resolution if any of the following are true:

- It has an `InputIterator` template parameter and a type that does not qualify as an input iterator is deduced for that parameter.
- It has an `Allocator` template parameter and a type that does not qualify as an allocator is deduced for that parameter.
- It has a `Compare` template parameter and a type that qualifies as an allocator is deduced for that parameter.
- It has an `Alloc` template parameter and `uses_allocator_v<X, Alloc>` is false where `X` is the deduced type of the associative container.

(We think that change is not strictly necessary, but it clarifies our intent.)

Add one new deduction guide for each associative container. For example, modify [\[set.overview\]](#) as follows:

```

template<class InputIterator,
        class Compare = less<iter-value-type<InputIterator>>,
        class Allocator = allocator<iter-value-type<InputIterator>>>
set(InputIterator, InputIterator,
    Compare = Compare(), Allocator = Allocator())
-> set<iter-value-type<InputIterator>, Compare, Allocator>;

template<class Key, class Compare = less<Key>, class Allocator = allocator<Key>>
set(initializer_list<Key>, Compare = Compare(), Allocator = Allocator())
-> set<Key, Compare, Allocator>;

template<class InputIterator, class Allocator>
set(InputIterator, InputIterator, Allocator)
-> set<iter-value-type<InputIterator>,
    less<iter-value-type<InputIterator>>, Allocator>;

template<class Key, class Allocator>
set(initializer_list<Key>, Allocator) -> set<Key, less<Key>, Allocator>;

template<class Key, class Compare, class Allocator, class Alloc>
set(set<Key, Compare, Allocator>, Alloc) -> set<Key, Compare, Allocator>;

```

Then, modify [\[unord.req\]/18](#) as follows:

A deduction guide for an unordered associative container shall not participate in overload resolution if any of the following are true:

- It has an `InputIterator` template parameter and a type that does not qualify as an input iterator is deduced for that parameter.
- It has an `Allocator` template parameter and a type that does not qualify as an allocator is deduced for that parameter.
- It has a `Hash` template parameter and an integral type or a type that qualifies as an allocator is deduced for that parameter.
- It has a `Pred` template parameter and a type that qualifies as an allocator is deduced for that parameter.
- It has an `Alloc` template parameter and `uses_allocator_v<X, Alloc>` is false where `X` is the deduced type of the associative container.

(We think that change is not strictly necessary, but it clarifies our intent.)

Add one new deduction guide for each unordered associative container. For example, modify [\[unord.set.overview\]](#) as follows:


```

template<class InputIterator,
        class Hash = hash<iter-value-type<InputIterator>>,
        class Pred = equal_to<iter-value-type<InputIterator>>,
        class Allocator = allocator<iter-value-type<InputIterator>>>
unordered_set(InputIterator, InputIterator, typename see below::size_type = see below,
             Hash = Hash(), Pred = Pred(), Allocator = Allocator())
-> unordered_set<iter-value-type<InputIterator>,
               Hash, Pred, Allocator>;

template<class T, class Hash = hash<T>,
        class Pred = equal_to<T>, class Allocator = allocator<T>>
unordered_set(initializer_list<T>, typename see below::size_type = see below,
             Hash = Hash(), Pred = Pred(), Allocator = Allocator())
-> unordered_set<T, Hash, Pred, Allocator>;

template<class InputIterator, class Allocator>
unordered_set(InputIterator, InputIterator, typename see below::size_type, Allocator)
-> unordered_set<iter-value-type<InputIterator>,
               hash<iter-value-type<InputIterator>>,
               equal_to<iter-value-type<InputIterator>>,
               Allocator>;

template<class InputIterator, class Hash, class Allocator>
unordered_set(InputIterator, InputIterator, typename see below::size_type,
             Hash, Allocator)
-> unordered_set<iter-value-type<InputIterator>, Hash,
               equal_to<iter-value-type<InputIterator>>,
               Allocator>;

template<class T, class Allocator>
unordered_set(initializer_list<T>, typename see below::size_type, Allocator)
-> unordered_set<T, hash<T>, equal_to<T>, Allocator>;

template<class T, class Hash, class Allocator>
unordered_set(initializer_list<T>, typename see below::size_type, Hash, Allocator)
-> unordered_set<T, Hash, equal_to<T>, Allocator>;

template<class T, class Hash, class Pred, class Allocator, class Alloc>
unordered_set(set<T, Hash, Pred, Allocator>, Alloc)
-> unordered_set<T, Hash, Pred, Allocator>;

```

§ 2.4. Implementation note

While implementers may of course implement this proposal exactly as described in the wording options above, we wish to point out that a number of library implementations already behave in many

cases as we are proposing here as shown in the following table and [this Godbolt](#):

<i>Construct</i>	<i>Well-formed</i>	<i>SFINAE-friendly ill-formed</i>	<i>Hard error</i>
<code>std::deque(pd, &mr)</code>		MSVC, libstdc++, libc++	
<code>std::forward_list(pfl, &mr)</code>		MSVC, libstdc++, libc++	
<code>std::list(pl, &mr)</code>		MSVC, libstdc++, libc++	
<code>std::vector(pv, &mr)</code>		MSVC, libstdc++, libc++	
<code>std::map(pm, &mr)</code>	MSVC	libstdc++, libc++	
<code>std::multimap(pmm, &mr)</code>	MSVC	libstdc++, libc++	
<code>std::multiset(pms, &mr)</code>	MSVC	libstdc++, libc++	
<code>std::set(ps, &mr)</code>	MSVC	libstdc++, libc++	
<code>std::unordered_map(pum, &mr)</code>	MSVC, libstdc++	libc++	
<code>std::unordered_multimap(pumm, &mr)</code>	MSVC, libstdc++	libc++	
<code>std::unordered_multiset(pums, &mr)</code>	MSVC, libstdc++	libc++	
<code>std::unordered_set(pus, &mr)</code>	MSVC, libstdc++	libc++	
<code>std::priority_queue(ppq, &mr)</code>	MSVC, libstdc++, libc++		
<code>std::queue(pq, &mr)</code>	MSVC, libstdc++, libc++		
<code>std::stack(ps, &mr)</code>	MSVC, libstdc++, libc++		
<code>std::priority_queue(pv, &mr)</code>		MSVC, libstdc++, libc++	
<code>std::queue(pd, &mr)</code>		MSVC, libstdc++, libc++	
<code>std::stack(pv, &mr)</code>		MSVC, libstdc++, libc++	

The following analysis explains why MSVC, in effect, already implements the approach described in [wording option 1](#) above. Consider the implicit guide generated from the constructor

```
template<class Key, class Hash, class Pred, class Allocator>
class unordered_set {
    unordered_set(const unordered_set<Key, Hash, Pred, Allocator>&,
                  const Allocator&);
};
```

From the first parameter, we deduce `Key=int`, `Hash=std::hash<int>`, `Pred=std::equal_to<int>`, and `Allocator=std::pmr::polymorphic_allocator<int>`. From the second parameter, we should deduce `Allocator=std::monotonic_buffer_resource*`. We've deduced conflicting types for `Allocator`, so deduction fails.

However, in practice, both MSVC and libstdc++ launder the `Allocator` type through a typedef that is opaque to deduction, so that the constructor seen by the compiler in practice is equivalent to

```
template<class Key, class Hash, class Pred, class Allocator>
class unordered_set {
    using allocator_type = typename type_identity<Allocator>::type;
    unordered_set(const unordered_set<Key, Hash, Pred, Allocator>&,
                  const allocator_type&);
};
```

From the first parameter, we deduce `Key=int`, `Hash=std::hash<int>`, `Pred=std::equal_to<int>`, and `Allocator=std::pmr::polymorphic_allocator<int>`. From the second parameter, we don't deduce anything. In practice, therefore, deduction succeeds.

Implementers who have taken the above approach can use that to simplify their implementation of this proposal, as an alternative to directly implementing the above wording (which is of course also perfectly fine).